Build the staff workspace.

Revised Set B - Vue frontend, supplied XAMPP backend, registration and login

GearDesk rents equipment for meetings and events. Staff need to register, log in, and manage one shared equipment inventory. Build the frontend using the supplied API and visual references.

Assessment window	Supplied materials
5 h 30 min total	30 min preparation and 5 h implementation for this rehearsal.
Backend provided	PHP API, backend .env, SQL database, API contract and smoke-check script.
Frontend required	Vue source, responsive UI, API integration, validation, tests, README and Git history.

Client requirements

- Register an account with name, email, password and confirmation. Switch to Login after success.
- Log in, read the returned access token, and attach it to every protected API request.
- Verify a saved session before revealing inventory after reload. Handle 401 and Logout correctly.
- List, add, edit and delete equipment. Populate category dropdowns from the API.
- Use server pagination (five per page), name search, category filter and clear boundary controls.
- Match the desktop references and provide mobile layouts, visible errors and keyboard access.

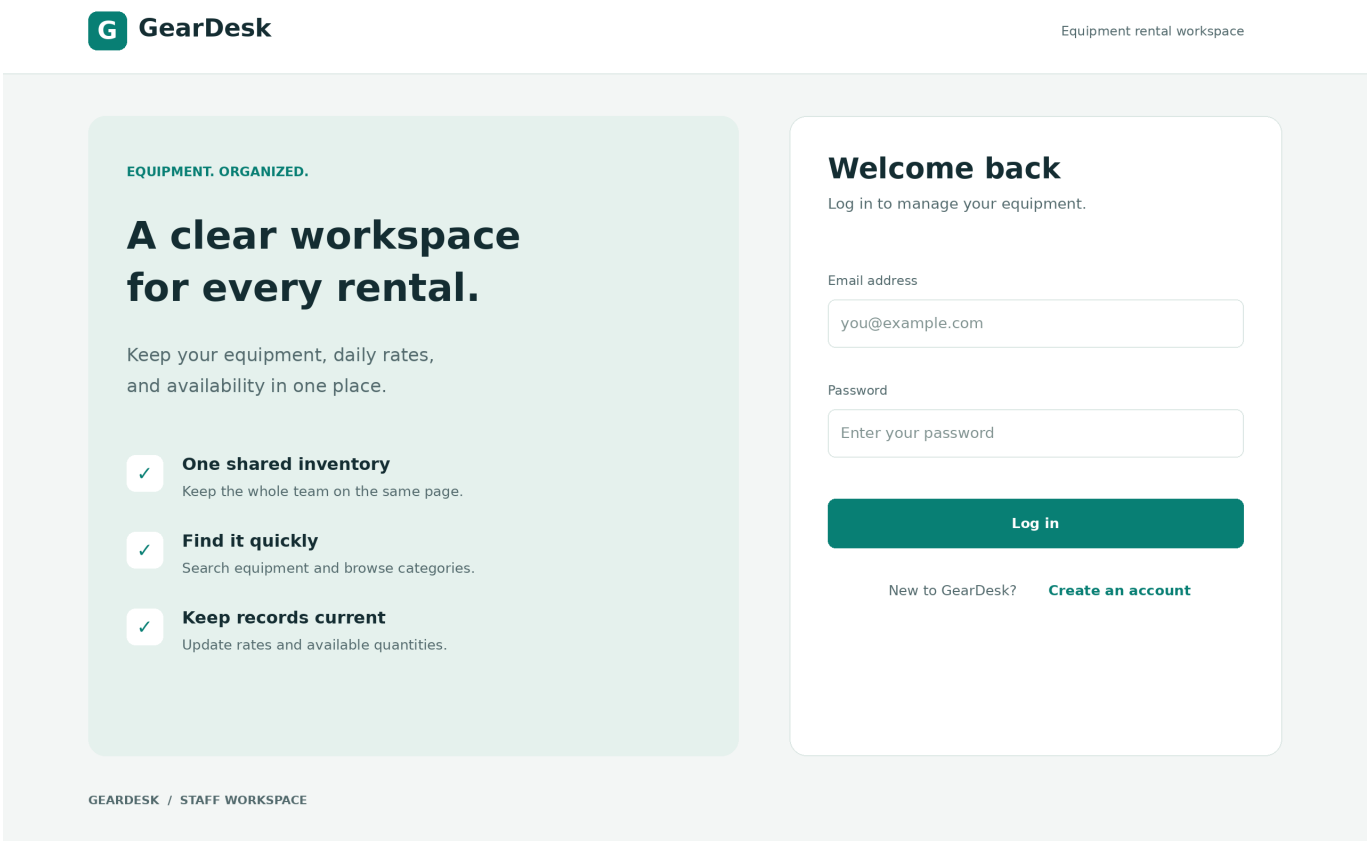
Learn first, then attempt the timer

Start with LEARN-AUTH-FIRST.md for small authentication exercises. The actual PHP backend is in xampp/geardesk-api/ and the SQL is database/geardesk.sql. Full requirements and API examples are in the accompanying Markdown files.

Original training exercise. Authentication reflects participant reports; exact assessment endpoints, token format and task details remain subject to the supplied assessment instructions.

Welcome back.

Desktop target: 1280px. Do not display token strings in the finished interface.



Required behavior

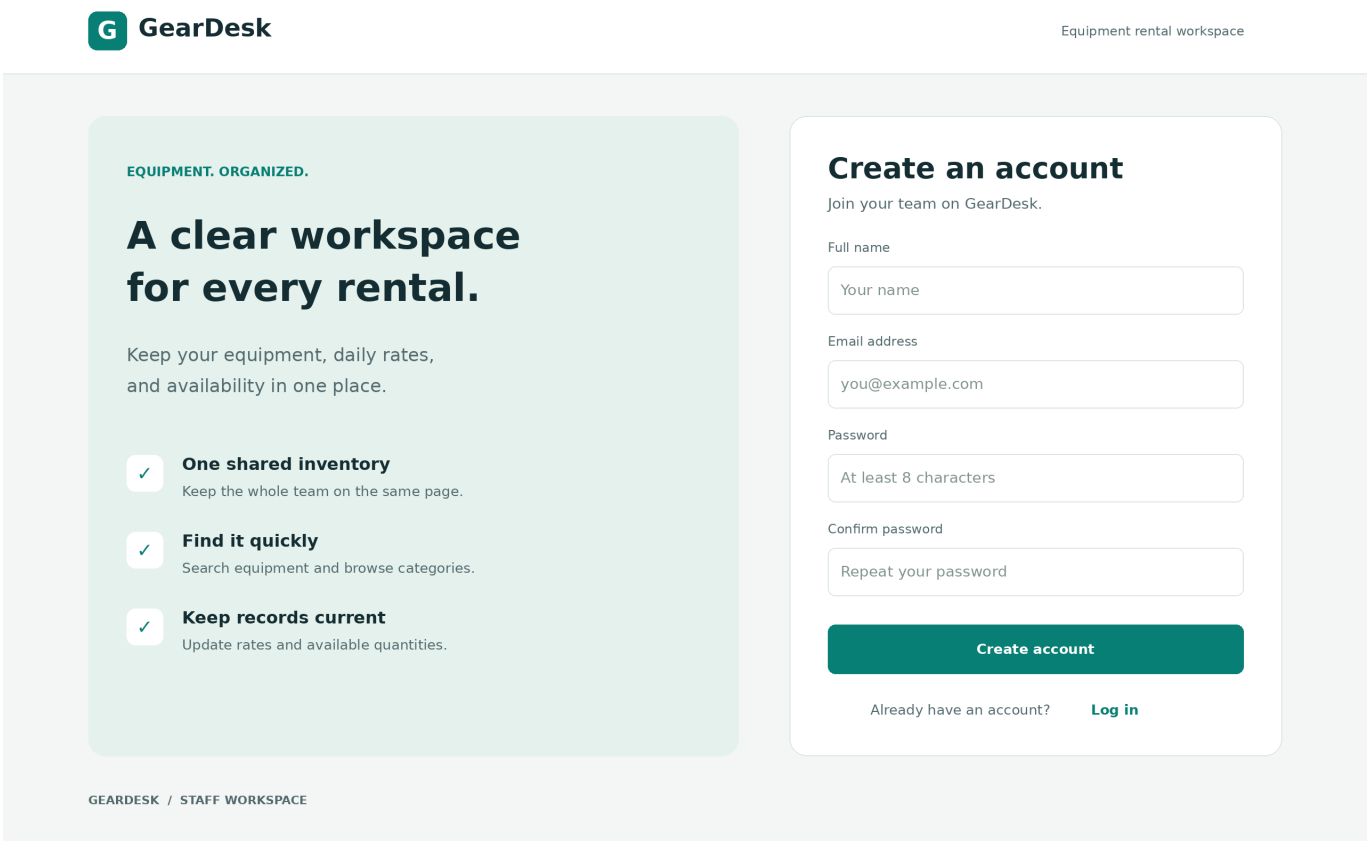
Submit email and password to POST ?resource=login. Wrong credentials remain on Login with a useful message. On success, read data.access_token and data.user, then load the signed-in workspace.

Login response field	Purpose
data.access_token	Send this value in Authorization: Bearer <token>.
data.token_type	Bearer
data.expires_in	Lifetime in seconds; normally 3600.
data.user	Current user id, name and email.

A token is returned by the backend. You do not generate one or use the MySQL password as a token. Passwords belong only in the login/registration request, not browser storage.

Create your account.

Registration is separate from Login in this supplied practice API.

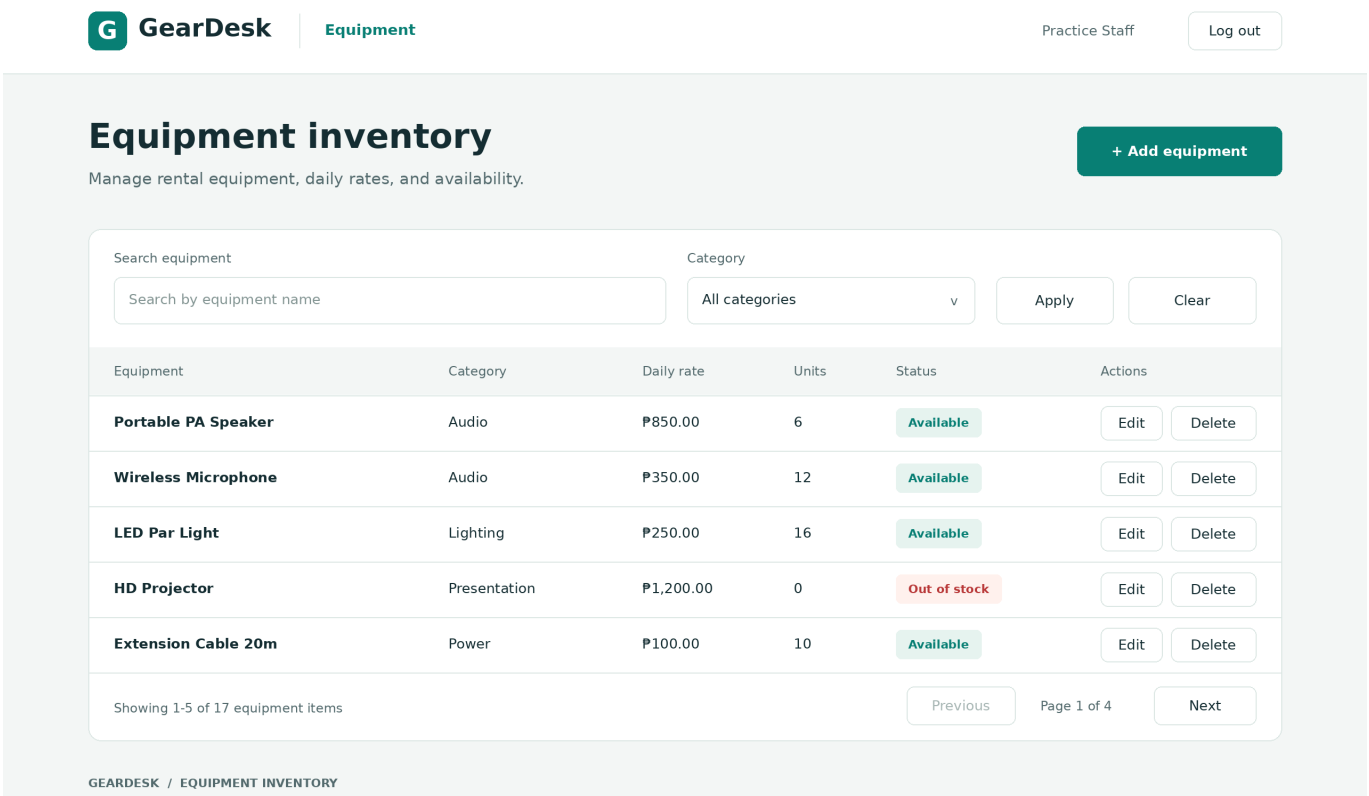


Field	Validation
Full name	Trimmed, required, 1-80 characters.
Email address	Valid, at most 190 characters, unique.
Password	8-72 bytes; use ASCII for this exercise.
Confirm password	Must match the password exactly.

POST ?resource=register returns 201 and the created user, with no token. Show success and switch to Login. Display 409 for an existing email and 422 for invalid fields. Prevent duplicate submissions while pending.

The signed-in inventory.

Current user and Logout stay visible in the header.

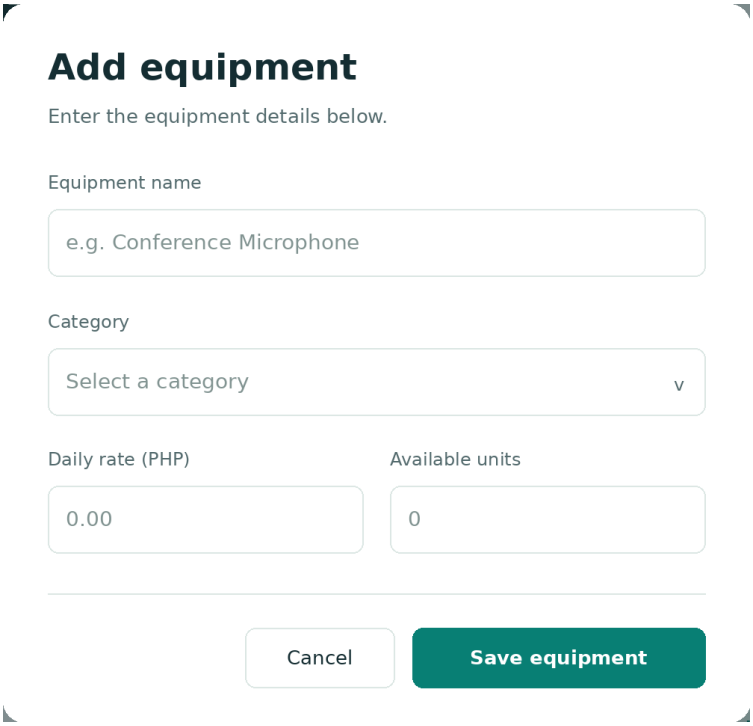


Region	Behavior
Header	Brand, Equipment navigation, current user, Logout.
Toolbar	Name search, category filter, Apply, Clear. Reset to page 1 on filter changes.
Records	Name, category, peso daily rate, units, derived status, Edit/Delete.
Footer	Current range and total, Previous, Page X of Y, Next.
Fresh dataset	17 records, four categories, pages of 5 / 5 / 5 / 2.

The API already paginates its response. Use result.data for rows and result.meta for page totals; do not slice those returned rows again. A zero-result search needs an empty state, not Page 1 of 0.

Keep the same task usable.

One dialog of each type, outside record loops. Mobile references are included separately.



Field	Rule
Equipment name	Required, trimmed, 1-80 characters.
Category	Numeric ID selected from the categories API.
Daily rate	Number greater than 0, maximum 999999.99, at most two decimal places.
Available units	Integer 0-9999. Zero is valid and means Out of stock.

Edit all four fields. Cancel preserves the original record. Close only after a successful save; failed requests keep ordinary inputs and show useful errors. Confirm Delete with the equipment name. Handle success 204 without JSON parsing.

At 375px and 320px, use readable inventory cards and full-width auth forms. Provide labels, visible focus, modal focus management, Escape/Cancel behavior and focus return. See mobile.png, login-mobile.png and register-mobile.png.

Read, then connect.

API base: http://localhost/geardesk-api/index.php

Method / route	Auth	Response
GET ?resource=health	No	200; database connected, authentication bearer.
POST ?resource=register	No	201; created user in data, no token.
POST ?resource=login	No	200; data.access_token, token_type, expires_in, user.
GET ?resource=me	Yes	200; current user in data.
POST ?resource=logout	Yes	200; current token revoked.

Authentication flow

1. Registration sends name, email, password and password_confirmation.

2. Login sends email and password; store data.access_token in Vue state.

3. Protected requests send Authorization: Bearer followed by the raw token.

4. For this local exercise, sessionStorage may retain the token through reloads. Verify it using me before showing inventory.

5. Protected-request 401 clears the session. Network/503 failures show Retry and do not prove the token invalid.

6. Logout revokes the server token, then clears local state/storage and private data.

Different credentials, different jobs

Value	Location / use
MySQL user/password	Backend .env; PHP connects to the database.
User email/password	Registration and Login forms.
Access token	Authorization header on protected API calls.

This API uses opaque tokens, not JWTs. There is no refresh-token endpoint. All users share the practice inventory. Use the exact response fields in API-DOCUMENTATION.md.

The CRUD you know, with a header.

All endpoints on this page require Authorization: Bearer <token>.

Method	Append to API base	Success
GET	?resource=categories	200; data array
GET	?resource=equipment&page=1&per_page=5	200; data + meta
GET	?resource=equipment&id=1	200; one object in data
POST	?resource=equipment	201; created object in data
PATCH	?resource=equipment&id=1	200; updated object in data
DELETE	?resource=equipment&id=1	204; empty body

JSON save fields

Send name, category_id, daily_rate and available_units. Numeric values must be JSON numbers. IDs are generated by the backend; category_name is read-only. PATCH accepts any nonempty subset of those four writable fields.

Pagination and filtering

Query fields: page, per_page, optional search, optional category_id. Response metadata: page, per_page, total, total_pages. Omit category_id for All categories. Search and category combine. After deleting the last record on a page, request a valid preceding page.

Error handling

Status	Action
401	On Login: wrong credentials. On protected calls: clear session and show Login.
409 / 422	Show duplicate-email or validation message/field errors.
404 / 415	Check record/route or JSON Content-Type.
503 / network failure	Display a useful message and Retry; preserve ordinary inputs.

fetch() does not throw automatically for HTTP failures: inspect response.ok/status. POST and PATCH need Content-Type: application/json as well as Authorization.

Finish a complete submission.

Use MANUAL-TESTS.csv to record actual results after the attempt.

Implementation time	Target
0:00-0:55	Registration/Login, token handling, protected GET, Logout
0:55-1:40	Inventory layout, category loading, display states
1:40-2:40	Add/Edit modals, validation, authenticated saves
2:40-3:05	Delete confirmation and authenticated deletion
3:05-3:45	Server pagination, search and category filter
3:45-4:35	Responsive, session, persistence and failure tests
4:35-5:00	Fixes, README, test results, Git, build

Practice rubric	Points
Registration and Login	20
Token use, session verification, 401 handling, Logout	15
Inventory CRUD and categories	25
Server pagination, search and filter	15
Responsive UI and accessibility	10
Validation and failure/display states	10
Documentation, test evidence, Git and build	5
Total	100

Submit Vue source, manifest/lockfile, public .env.example, setup README, plan, test results and Git history. Exclude node_modules, passwords and token values. Practice target: 80/100 with auth, protected CRUD and pagination working; this is not an official competency threshold.

Before the timer: run the README health check and optional verify-api.ps1 on XAMPP. See VERIFICATION.md for checks performed here and runtime limits.